# TP1 — Architecture SOA et services Web

**Domaine choisi : Gestion d'un magasin de produits cosmétiques**

## 1. Création du document XML

Pour ce TP, j'ai choisi le domaine de la gestion d'un magasin de produits cosmétiques. L'objectif est de représenter les informations du magasin, ses catégories et ses différents produits dans un document XML.

Le fichier `produits.xml` contient trois parties principales :

- **Informations :** contient les informations générales du magasin, comme son nom, sa ville, son adresse et son numéro de téléphone.
- **Catégories :** contient les différentes catégories de produits disponibles dans le magasin.
- **Produits :** contient les informations détaillées de chaque produit.

Chaque produit possède un identifiant, un nom, une marque, un prix, une quantité en stock, une indication de promotion et une référence vers sa catégorie.

Le document contient cinq produits : Crème hydratante, Shampooing, Parfum, Crème solaire et Masque capillaire.

Cette organisation permet de structurer les données et de faciliter leur consultation.

### Structure du document XML

Le fichier `produits.xml` utilise l'élément `<magasin>` comme élément racine. Il contient les éléments `<informations>`, `<categories>` et `<produits>`.

Exemple de structure :

```
<?xml version="1.0" encoding="UTF-8"?>
<magasin>
  <informations>
    <nom>Beauty Store</nom>
    <ville>Sfax</ville>
    <adresse>Centre Ville</adresse>
    <telephone>74123456</telephone>
  </informations>
  <categories>
    <categorie id="C1">
      <nom>Soins du visage</nom>
```

```
        <description>Produits pour le soin du visage</description>
    </categorie>
    <categorie id="C2">
        <nom>Soins capillaires</nom>
        <description>Produits pour les cheveux</description>
    </categorie>
    <categorie id="C3">
        <nom>Parfumerie</nom>
        <description>Parfums et eaux de toilette</description>
    </categorie>
</categories>
<produits>
    <produit id="P1" categorieRef="C1">
        <nom>Creme hydratante</nom>
        <marque>Nivea</marque>
        <prix>25.500</prix>
        <stock>40</stock>
        <promotion>true</promotion>
    </produit>
    <produit id="P2" categorieRef="C2">
        <nom>Shampooing</nom>
        <marque>Pantene</marque>
        <prix>18.000</prix>
        <stock>25</stock>
        <promotion>false</promotion>
    </produit>
    <produit id="P3" categorieRef="C3">
        <nom>Parfum</nom>
        <marque>Chanel</marque>
        <prix>80.000</prix>
        <stock>15</stock>
        <promotion>true</promotion>
    </produit>
```

```
        <produit id="P4" categorieRef="C1">
            <nom>Creme solaire</nom>
            <marque>La Roche Posay</marque>
            <prix>45.000</prix>
            <stock>30</stock>
            <promotion>false</promotion>
        </produit>
        <produit id="P5" categorieRef="C2">
            <nom>Masque capillaire</nom>
            <marque>Garnier</marque>
            <prix>32.000</prix>
            <stock>20</stock>
            <promotion>true</promotion>
        </produit>
    </produits>
</magasin>
```

## 2. Définition du schéma XSD

J'ai créé le fichier `produits.xsd` afin de définir les règles et la structure que le document XML doit respecter.

Le schéma XSD permet de vérifier les éléments, leurs types de données, leurs valeurs et leur ordre.

Dans ce TP, plusieurs contraintes sont définies :

- Le nom du magasin et les noms des produits doivent être des textes valides.
- Le numéro de téléphone doit contenir exactement 8 chiffres.
- Le prix doit être un nombre décimal strictement positif et ne doit pas dépasser 10000.
- La quantité en stock doit être un entier positif ou égal à zéro.
- La promotion doit avoir une valeur booléenne ( `true` ou `false` ).
- Chaque catégorie possède un identifiant unique.
- Chaque produit possède un identifiant et une référence vers une catégorie existante.
- Les éléments doivent respecter l'ordre défini dans le schéma.

L'utilisation du schéma XSD permet donc d'éviter les erreurs dans la structure et les données du document XML.

## 3. Implémentation de la validation XML/XSD

Pour réaliser la validation, j'ai utilisé le langage Java dans IntelliJ IDEA avec un projet Maven.

Dans le fichier `Main.java`, j'ai utilisé les classes suivantes :

- `SchemaFactory` : permet de créer un schéma XSD.
- `Schema` : représente le schéma de validation.
- `Validator` : permet de vérifier si le document XML respecte les règles du schéma.
- `XMLConstants` : permet de préciser le langage de schéma XML utilisé.

Le programme récupère les fichiers `produits.xml` et `produits.xsd` depuis le dossier des ressources du projet, puis effectue la validation.

J'ai également créé un deuxième fichier nommé `produits-invalide.xml` contenant une erreur volontaire : un prix négatif. Ce fichier permet de vérifier que le schéma XSD détecte correctement les données incorrectes.

Ainsi, le programme réalise deux tests :

1. Validation du fichier `produits.xml` : le document doit être accepté car il respecte les règles du schéma.
2. Validation du fichier `produits-invalide.xml` : le document doit être refusé à cause du prix négatif.

Cette étape permet de vérifier que le schéma XSD fonctionne correctement.

## 4. Interrogation XPath

J'ai utilisé XPath pour rechercher et extraire des informations à partir du document XML.

Les expressions XPath permettent de sélectionner des éléments selon différents critères, sans parcourir manuellement tout le document.

Dans le fichier `Main.java`, j'ai utilisé `XPathFactory` pour créer un objet XPath et `XPathConstants.NODESET` pour récupérer les résultats sous forme de liste de nœuds.

J'ai réalisé les huit requêtes suivantes :

**Requête 1 : Afficher les noms de tous les produits**

```
/magasin/produits/produit/nom
```

Cette requête récupère les noms de tous les produits du magasin.

**Requête 2 : Afficher les produits en promotion**

```
/magasin/produits/produit[promotion='true']/nom
```

Cette requête sélectionne uniquement les produits dont la promotion est égale à `true`.

**Requête 3 : Afficher les produits dont le prix est supérieur à 30**

```
/magasin/produits/produit[prix > 30]/nom
```

Cette requête permet de retrouver les produits dont le prix dépasse 30.

**Requête 4 : Afficher les produits dont le stock est inférieur à 30**

```
/magasin/produits/produit[stock < 30]/nom
```

Cette requête sélectionne les produits dont la quantité en stock est inférieure à 30.

**Requête 5 : Afficher les produits appartenant à la catégorie C1**

```
/magasin/produits/produit[@categorieRef='C1']/nom
```

Cette requête récupère les produits associés à la catégorie C1, qui correspond aux soins du visage.

**Requête 6 : Afficher les marques des produits**

```
/magasin/produits/produit/marque
```

Cette requête permet d'afficher les différentes marques des produits.

**Requête 7 : Afficher les noms des catégories**

```
/magasin/categories/categorie/nom
```

Cette requête récupère les noms de toutes les catégories du magasin.

**Requête 8 : Afficher les produits dont le stock est égal à zéro**

```
/magasin/produits/produit[stock=0]/nom
```

Cette requête permet d'identifier les produits qui ne sont plus disponibles en stock.

## 5. Test et vérification

Après l'exécution de la méthode `main()` dans IntelliJ IDEA, le programme effectue les opérations suivantes :

- Il valide le document XML principal avec le schéma XSD.
- Il teste le document XML invalide afin de vérifier la détection des erreurs.
- Il charge le document XML.
- Il exécute les différentes requêtes XPath.
- Il affiche les résultats dans la console.

### Résultat attendu de la validation

Pour le fichier `produits.xml`, le résultat attendu est :

```
Validation reussie : produits.xml
```

Pour le fichier `produits-invalide.xml`, le programme doit signaler une erreur de validation à cause du prix négatif.

### Résultats attendus des requêtes XPath

```
        INTERROGATION XPATH
==================================

1. Tous les noms des produits :
- Creme hydratante
- Shampooing
- Parfum
- Creme solaire
- Masque capillaire

2. Les produits en promotion :
- Creme hydratante
- Parfum
- Masque capillaire

3. Les produits dont le prix depasse 30 :
- Parfum
- Creme solaire
- Masque capillaire

4. Les produits avec un stock inferieur a 30 :
- Shampooing
- Parfum
- Masque capillaire

5. Les produits de la categorie C1 :
- Creme hydratante
- Creme solaire
```

```
- Masque capillaire

4. Les produits avec un stock inferieur a 30 :
- Shampooing
- Parfum
- Masque capillaire

5. Les produits de la categorie C1 :
- Creme hydratante
- Creme solaire

6. Les marques des produits :
- Nivea
- Pantene
- Chanel
- La Roche Posay
- Garnier

7. Les categories du magasin :
- Soins du visage
- Soins capillaires
- Parfumerie

8. Les produits sans stock :
Aucun resultat trouve.
```

## Conclusion

Ce TP m'a permis de découvrir et de pratiquer plusieurs technologies liées à la manipulation des données XML.

J'ai appris à créer un document XML structuré pour représenter les informations d'un magasin de produits cosmétiques, à définir un schéma XSD pour contrôler la structure et les valeurs des données, ainsi qu'à utiliser Java pour effectuer la validation des documents XML.

J'ai également utilisé XPath pour rechercher et extraire différentes informations selon plusieurs critères.

Enfin, le test d'un document XML valide et d'un document XML invalide m'a permis de vérifier le fonctionnement de la validation XSD.

Ce travail m'a permis de mieux comprendre l'utilisation de XML, XSD, XPath et Java dans le traitement et l'organisation des données.